

Supplementary Material

Monopath DAGs: Structured Patient Trajectories from Clinical Case Reports

Code, graphs, prompts, and analysis scripts: <https://github.com/DaneshjouLab/MonopathDAGs>.

Contents

S1 Notations and Definitions	2
S1.1 Task definition	2
S1.2 Notation for patient trajectory graphs	2
S2 Additional Data	4
S2.1 Structure and semantics of DAGs	4
S2.2 Rare disease clustering output	4
S3 Output Overview	5
S3.1 Node and edge attributes	5
S3.2 Node output example	5
S3.3 Edge output example	5
S3.4 DAG construction algorithm	7
S4 Human Evaluation	8
S4.1 Human evaluation schematic	8
S4.2 Fidelity Evaluation	8
S4.3 Clinical Evaluation	9
S5 Prompt Design	10
S5.1 Narrative-generation prompts	10
S5.2 Graph-construction signatures	10
S5.2.1 Shared primer (<code>dag_primer</code>)	10
S5.2.2 Patient timeline	11
S5.2.3 Node construction	11
S5.2.4 Atomic decomposition	12
S5.2.5 Clinical-data extraction	13
S5.2.6 Edge construction	14
S6 Additional Related Work	16
S6.1 Expansion beyond existing extraction methods	16
S7 Computational Resources	17
S8 Physician evaluation: analyses	18
S9 Clustering and metastasis analyses	20

S1 Notations and Definitions

S1.1 Task definition

We focus on the monopath constraint—each node has in-degree ≤ 1 and out-degree ≤ 1 , and the graph is connected—to establish feasibility of robust, large-scale extraction and to preserve temporal directionality. More expressive graph structures (*e.g.*, multiplex or hypergraphs) can capture concurrent or cyclic phenomena but complicate interpretability. We view them as complementary extensions (see Limitations).

$$\mathcal{D} = \{d_1, d_2, \dots, d_N\} \quad (1)$$

denotes a corpus of free-text clinical case reports, where each d_i describes a longitudinal patient narrative. For each report d_i , our objective is to construct a directed acyclic graph (DAG)

$$G_i = (V_i, E_i) \in \mathcal{G}, \quad i = 1, \dots, N, \quad (2)$$

that captures the patient’s clinical trajectory in a structured format.

Graph Representation. We denote a generic DAG as $G = (V, E)$, where

$$V = \{v_1, \dots, v_K\}, \quad E \subseteq V \times V. \quad (3)$$

Node Semantics. Each node $v_k \in V$ corresponds to a temporally anchored patient state and is annotated with:

$$\begin{aligned} s: V &\rightarrow \mathcal{T}, & s(v_k) &= s_k & (\text{free-text content}), \\ a: V &\rightarrow \mathcal{A}, & a(v_k) &= a_k & (\text{structured clinical data attributes}), \\ t: V &\rightarrow \mathbb{R}_{>0} \cup \{\emptyset\}, & t(v_k) &= t_k & (\text{optional timestamp}). \end{aligned} \quad (4)$$

The attribute set a_k typically includes symptoms, diagnoses, laboratory values, medications, and procedures.

Edge Semantics. Each edge $e_{ij} = (v_i \rightarrow v_j) \in E$ denotes a transition between two patient states. We define edge annotations as:

$$\begin{aligned} \tau: E &\rightarrow \mathcal{T}, & \tau(e_{ij}) &= \tau_{ij} & (\text{transition type}), \\ \phi: E &\rightarrow 2^{\mathcal{C}}, & \phi(e_{ij}) &= \varphi_{ij} & (\text{triggering clinical entities}), \end{aligned} \quad (5)$$

where $\mathcal{C}$ denotes a standardized clinical ontology [*e.g.*, Unified Medical Language System (UMLS)]. Thus, each G_i yields a machine-readable, temporally ordered abstraction of the unstructured narrative in d_i .

Event Standardization. To convert free-text narratives into machine-actionable graph elements, all events are normalized into ontology-grounded fields. Sentences are decomposed into atomic facts, and the `NodeClinicalDataExtract` module maps each fact to Unified Medical Language System (UMLS) concepts, yielding a structured `clinical_data` dictionary per node. Edges are similarly represented as typed JSON `transition_event` objects specifying trigger, domain, change type, and ontology-mapped entities ($\phi(e) \subseteq \mathcal{C}$, where $\mathcal{C}$ is the ontology). Narrative strings are retained only as evidence, while ontology grounding enables programmatic querying, aggregation, and embedding-based analysis.

S1.2 Notation for patient trajectory graphs

This table summarizes the notation for patient trajectory graphs, including case reports, DAGs, clinical state nodes, and directed transitions. Nodes may include free-text summaries, structured attributes, timestamps, and mapped clinical concepts, while edges capture labeled state transitions. Together, the notation provides a consistent framework for representing patient trajectories.

Table S1.1: Notation for patient trajectory graphs

Notation	Description
$D = \{d_1, d_2, \dots, d_N\}$	Corpus of free-text clinical case reports
$d_i \in D$	A single case report
$G_i = (V_i, E_i)$	DAG representing the patient trajectory for case d_i
$\mathcal{G}$	Set of all patient trajectory graphs
$V = \{v_1, \dots, v_K\}$	Set of nodes representing clinical states
$E \subseteq V \times V$	Set of directed edges between states
$v_k \in V$	A single patient state node
$s_k \in \mathcal{T}$	Free-text content of the patient state at v_k
$a_k \in \mathcal{A}$	Structured clinical data attributes at v_k
$t_k \in \mathbb{R}^+ \cup \{\emptyset\}$	Optional timestamp for node v_k
$e_{ij} = (v_i \rightarrow v_j) \in E$	Directed transition from v_i to v_j
$\tau_{ij} \in \mathcal{T}$	Transition type label for edge e_{ij}
$\phi_{ij} \subseteq \mathcal{C}$	Triggering clinical entities mapped to ontology
$\mathcal{T}$	Space of free-text summaries or transition types
$\mathcal{A}$	Space of structured clinical attributes
$\mathcal{C}$	Space of clinical concepts/entities

S2 Additional Data

S2.1 Structure and semantics of DAGs

Semantic accuracy is assessed using BERTScore (mean $\pm$ standard deviation) across precision, recall, and F1. Topological correctness is evaluated through graph-theoretic metrics: the percentage of acyclic graphs, the number of weakly connected components, and average node-level statistics (node count, edge count, in-degree, and graph density). These metrics quantify both structural validity and interpretability of the generated DAGs. Of the 485 graphs, 405 (83.5%) form a single connected path; 77 (15.9%) are broken into 2–10 segments because the model did not link some consecutive states; 2 branch; and 1 contains a cycle. A graph fails the connectivity check when pieces of the path are not linked to each other, which is why the mean number of weakly connected components exceeds one.

Metric	Monopath DAG
BERTScore (mean $\pm$ std; full texts, 510-token windows)	
Precision	0.807 ± 0.061
Recall	0.786 ± 0.042
F1 Score	0.795 ± 0.041
Topology Validation	
Acyclic	484/485 (99.8%)
Weakly Connected Component	1.436 ± 1.252
Avg. Node Count	8.182 ± 3.139
Avg. Edge Count	6.749 ± 2.567
Avg. In-Degree	0.820 ± 0.108
Graph Density	0.137 ± 0.068

S2.2 Rare disease clustering output

These values were computed with the original analysis configuration (512-token truncation) and are reported as exploratory.

Comparison of Graph vs. Text representations across embeddings. Values shown are (Graph, Text), with percent improvement of Graph over Text. Silhouette score measures how well-separated clusters are, with higher values indicating more coherent and distinct groupings. The Calinski–Harabasz (CH) index evaluates clustering compactness and separation, where larger values reflect tighter, well-defined clusters.

Dataset	Embedding	Silhouette		Calinski–Harabasz	
		Values	% Δ	Values	% Δ
Rare Disease	Raw	(0.1340, 0.0978)	+37.0%	(5.12, 4.23)	+21.0%
Rare Disease	PCA	(0.2450, 0.1777)	+37.9%	(15.63, 12.56)	+24.4%
Rare Disease	UMAP	(0.4845, 0.3727)	+29.9%	(103.25, 51.44)	+100.8%

S3 Output Overview

S3.1 Node and edge attributes

The following table lists the structured attributes used in our graph representation of patient trajectories. Nodes represent individual clinical states, while edges denote transitions between those states. Not every field is present in every case. Only available information from the narrative is populated.

Node Attributes	Edge Attributes
Node ID: unique identifier (e.g., “A”, “B”) in order of appearance.	Edge ID: concatenation of source and target node IDs (e.g., “A→B”).
Step Index: index in the sequence.	Narrative Content: description of the event or change.
Narrative Content: descriptive text of the state.	Transition Event: structured encoding with:
Timestamp: extracted if explicitly mentioned.	Trigger Type, Trigger Entities, Change Type,
Clinical Data: structured key-value fields (e.g., vitals, labs, medications, imaging, HPI, etc.).	Affected Domain

S3.2 Node output example

The following is an example of structured content for a node extracted from a patient case report. It includes narrative summary, structured clinical data (such as HPI and social history), and concept codes.

Node Details: graph_001, File: Intriguing_Encounter__Unveiling_Squamous_Cell_C2
• id: N1 • label: Step 1 • customData: – node_id: A – node_step_index: 0 – content: 44-year-old male presented with right-sided chest pain, dry cough, on-off fever, and hematuria for 2 months. Patient has no history of Antitubercular treatment (ATT) intake and was a tobacco chewer for >20 years. Physical examination revealed decreased air entry on the right side of the lung. – clinical_data: – HPI: – summary: right-sided chest pain, dry cough, on-off fever, and hematuria for 2 months – duration: 2 months – associated_symptoms: ◦ C0008031 ◦ C0010200 ◦ C0015967 ◦ C0019062 – social_history: – category: tobacco – status: current – description: tobacco chewer for >20 years • custom_id: graph_001_N0

S3.3 Edge output example

The following is an example of a structured edge output from a case, representing a transition between two clinical states. It includes both narrative summary and structured transition fields.

Edge Details: graph_001, File: Intriguing_Encounter__Unveiling_Squamous_Cell_Ca

- **from:** N1
- **to:** N2
- **data:**
 - **edge_id:** A_to_B
 - **content:** Initial presentation and subsequent imaging
 - **transition_event:**
 - **trigger_type:** imaging
 - **trigger_entities:**
 - **change_type:** addition
 - **target_domain:** imaging
- **custom_id:** graph_001_N1_N2
- **id:** 413a3142-63b0-4eeb-9830-26011a516e1a

S3.4 DAG construction algorithm

To illustrate how free-text case reports are systematically transformed into structured trajectory graphs, we provide the procedure below. The algorithm outlines the steps for segmenting narratives, constructing nodes and edges, extracting clinical data, and serializing the resulting Monopath DAG representation.

Algorithm 1 DSPy-Guided Monopath DAG Construction

```
1: Input: Free-text case report  $d_i$ 
2: Output: Monopath DAG  $G_i = (V_i, E_i)$ 
3:  $p_{1:k} \leftarrow \text{SEGMENT}(d_i, \text{window} = 10 \text{ sentences})$ 
4:  $T_i \leftarrow \text{PATIENTTIMELINE}(p_{1:k})$ 
5:  $V_i \leftarrow []$ 
6: for each block  $c_j$  in  $\text{CHUNK}(T_i, 4 \text{ sentences})$  do
7:    $v \leftarrow \text{NODECONSTRUCT}(c_j, \text{mem} = V_i)$ 
8:    $s^{\text{atom}} \leftarrow \text{DECOMPOSETOATOMICSENTENCES}(v.s)$ 
9:    $v.a \leftarrow \text{NODECLINICALDATAEXTRACT}(s^{\text{atom}})$ 
10:   $V_i \leftarrow \text{MERGEIFDUPLICATE}(v, V_i)$ 
11: end for
12:  $E_i \leftarrow []$ 
13: for  $t = 1$  to  $|V_i| - 1$  do
14:    $e \leftarrow \text{EDGECONSTRUCT}(V_i[t], V_i[t + 1])$ 
15:   append  $e$  to  $E_i$ 
16: end for
17:  $G_i \leftarrow \text{SERIALIZEASJSON}(V_i, E_i, \text{UMLS metadata})$ 
18: return  $G_i$ 
```

S4 Human Evaluation

S4.1 Human evaluation schematic

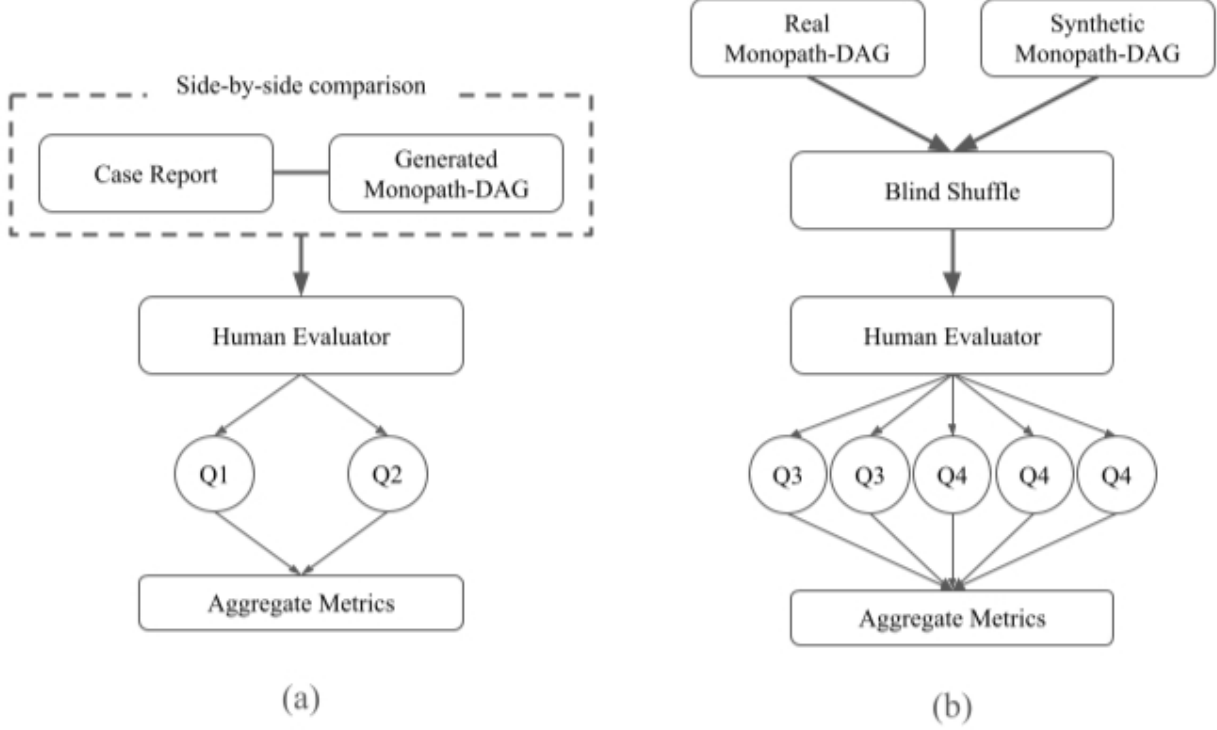


Figure S4.1: Human evaluation schematics for (a) fidelity and (b) physician ratings of synthetic narratives.

S4.2 Fidelity Evaluation

To assess whether generated Monopath-DAGs faithfully represent the underlying case reports, we implemented a two-part fidelity evaluation. First, we reconstructed narratives from graphs and compared them to original reports with BERTScore (ClinicalBERT embeddings). This provided automated precision, recall, and F1 measures of semantic similarity. Second, human annotators performed side-by-side comparisons of case reports and corresponding graphs, answering the following questions:

- **G1:** Are the nodes and edges in the correct order? [Yes / No]
- **G2:** Is the content of the graph accurate to the case report? [Yes / No]

Aggregate metrics from these assessments quantify structural and semantic fidelity (Figure S4.1a).

Graph review procedure. Two graduate students annotated the nodes and edges of 30 graphs against the source case report, answering G1 (order) and G2 (accuracy) for each element. The physician reviewed the graduate students' judgments and resolved disagreements; the final judgment is the one reported. A subset of 123 elements was annotated by both students; agreement was 94% for G1 and 92% for G2, and Cohen's κ was 0.73 for G1. We do not report κ for G2 because nearly all G2 ratings were positive, which makes κ uninformative.

Annotation guidelines.

- **G1 (order):** Answer “Yes” if the element is placed where it belongs in the patient’s course as described in the report: a node follows the states that precede it in the report and precedes those that follow it, and an edge connects an earlier state to a later one. Answer “No” if the element appears before an event that the report places earlier, or after one that the report places later.
- **G2 (accuracy):** Answer “Yes” if every clinical statement in the element (findings, values, diagnoses, treatments, timing) is supported by the case report. Answer “No” if the element contains content that contradicts or is absent from the report.
- **Partially correct content:** Answer “No” if any clinically relevant detail in that element is wrong or missing.

S4.3 Clinical Evaluation

To move beyond correctness, we conducted a clinician-centered evaluation focused on decision-making relevance. For each case, a graph-conditioned narrative and a control narrative (both synthetic) were shown to resident physicians (PGY-1 to PGY-3), blinded to condition and in random order within the case. Physicians saw only the narratives, not the graphs. They rated each narrative on five criteria, worded as follows:

- **Q1 (Clinical validity):** Are the case actions and insights medically sound?
- **Q2 (Timeline validity):** Does the case progression follow a clinically appropriate order?
- **Q3 (Clinical actionability):** Is the case specific and granular enough to guide the next clinical decision?
- **Q4 (Clinically safe action):** Is the information sufficient to make a safe decision with confidence?
- **Q5 (Appropriate language):** Does the narrative use medical language appropriately, avoiding generic or ambiguous phrasing?

The question text above is reproduced as shown to physicians; Q3 and Q4 are labeled “perceived” because they record the physicians’ judgment of the narrative, not an observed clinical outcome. Results are reported in Section S8.

S5 Prompt Design

This section reproduces the prompts used in the pipeline, copied from the code in the repository. Text is shown as it appears in the source, including spelling errors; lines are wrapped for width. Template placeholders (`{{ html }}`, `{{ path }}`, `{payload}`) are filled at run time. A few non-ASCII symbols in the source (arrows, check and cross marks) are typeset as the corresponding mathematical symbols.

S5.1 Narrative-generation prompts

The control arm extracts events directly from the case report HTML with one prompt and then writes a narrative from those events with a second prompt (`.config/prompts.ini`, sections `generate_control_nodes` and `path2text`; called in `generate_control_nodes` of `src/data/data_sets/generate_synthetic_case.py`). The graph-conditioned arm writes a narrative from the nodes and edges of one path through the Monopath DAG with the `LLMReconstructor` prompt template.

Control: event extraction (`prompts.ini`, `[generate_control_nodes]`)

```
Given the following HTML content, extract clinical events and return nodes:
{{ html }}
```

Control: narrative (`prompts.ini`, `[path2text]`)

```
Reconstruct the clinical case report from this data:\n\n
{{ path }} Write a coherent narrative including patient demographics, timeline of diagnoses, treatments, and outcomes.
Do not include anything else other than the full case
```

Graph-conditioned: narrative (`generate_synthetic_case.py`, `LLMReconstructor prompt_tpl`, line 383)

```
Reconstruct the clinical case report from this data:

{payload}

Write a coherent narrative including patient demographics, timeline of diagnoses, treatments, and outcomes. Do not include
↳ anything else other than the full case.
```

The `\n` in the control narrative prompt appears literally in the INI file. The reconstruction prompt used for the BERTScore fidelity evaluation is the graph-conditioned narrative prompt above, using the default template of `LLMReconstructor` (`src/benchmark/modules/reconstruction.py`), which omits the final sentence (“Do not include anything else other than the full case.”). The graph arm uses more model passes than the control (timeline, node construction, node reorganization, clinical-data extraction, edge construction, branch classification, and narrative writing, versus event extraction and narrative writing for the control).

S5.2 Graph-construction signatures

The graph is built with DSPy signatures (`src/agent/dspy/graph_generation.py` and `src/agent/dspy/testing_more.py`). DSPy sends each signature’s docstring as the task instruction and each field’s `desc` as the description of that input or output. For the node, edge, and clinical-data signatures, the docstring is set at run time as the shared `dag_primer` followed by the task-specific instruction from `docstring_dict`; we show the primer once.

S5.2.1 Shared primer (`dag_primer`)

`docstring_dict[dag_primer]`

```
You are an assistant that converts clinical case narratives into dynamic Directed Acyclic Graphs (DAGs).

Each DAG consists of:
- Nodes = snapshots of the patient's state.
- Edges = transitions between those states.

Terminology guidance:
- Use UMLS-standard concepts when possible for consistency and interoperability.
```

- If a concept isn't covered by UMLS, use clear, logical labeling.

Text extraction guidance:

- When looking at the case report input, ignore the references, background, conclusions etc. sections
- We only want to extract on content relating to the specific patient discussed in the case report

S5.2.2 Patient timeline

Applied paragraph by paragraph with the last three timeline outputs as memory (generate_patient_timeline); the final output is the cleaned case text passed to node construction.

PatientTimeline (testing_more.py, line 710)

```
DOCSTRING:
Your task is to extract structured patient timeline from a clinical paragraph. First evaluate if the information is part of
↳ the clinical case, or just introduction.
Your goal is to verify that it is a part of the clinical case and keep patient management and care in order.
your goal is to include everything including lab findings
INCLUDE ALL LAB FINDINGS VERBATIM AND ALL IMAGING FINDINGS VERBATIM
ONLY INCLUDE PATIENT DATA
DO NOT BULLET POINT, this is a str output, keep specific lab and imaging in as well as procedure values. No explanations

FIELDS:
paragraph: str = dspy.InputField(desc="Paragraph of clinical text to extract patient information from.")
previous_memory: List[str] = dspy.InputField(desc="List of patient history extracted so far.")
pt_timeline: str = dspy.OutputField(
    desc="Concatenated highly specific human-readable patient timeline including new events extracted in order."
)
```

S5.2.3 Node construction

Applied to successive four-sentence chunks of the timeline text, with the nodes generated so far passed as node_memory. Docstring = dag_primer + node_instructions.

nodeConstruct (graph_generation.py, line 271): node_instructions and fields

```
DOCSTRING:
You are given a clinical case report. Your task is to extract a sequence of nodes representing the patient's evolving clinical
↳ state.

Guidelines:
- Create one node per clinically meaningful state.
- Combine co-occurring labs/imaging into the same node.
- Use separate nodes for clearly sequential or distinct events.
- Do not return anything outside the list format. Should be in JSON compatible style.
- Keep imaging content packaged in one node if no clear temporal change is indicated
- Keep pathology / histology content packaged in one node if no clear temporal change is indicated
- node_memory is a running memory that updates as we add new nodes; use it to preserve context, merge overlapping details, and
↳ avoid redundant or stale states

Output format:
Return a list of node dictionaries, in this order from top to bottom, each with:
- node_id (In ascending alphabetical order, e.g., "A", "B", "C")
- node_step_index (integer for order)
- content (exhaustive clinical content, include all relevant details for the given node)
- timestamp (optional, ISO8601)

Example:
[±
{
  "node_id": "A",
  "node_step_index": 0,
  "content": "The patient presented with bilateral painless testicular masses."
}
]

FIELDS:
text_input: str = dspy.InputField(desc="body of text extracted from a case report")
node_memory: list[dict] = dspy.InputField(optional=True, desc="List of nodes generated so far (memory). Use as context when
↳ generating new nodes so there is no duplication.")
node_output = dspy.OutputField(type=list[dict], desc="A list of node dictionaries with node_id, node_step_index, content,
↳ optional timestamp and clinical_data")
```

After all chunks are processed, one further pass merges and re-indexes the nodes.

ReorganizeNodes (graph_generation.py, line 670)

```
DOCSTRING:
Take a full list of extracted nodes, deduplicate or merge any
overlapping/fragmented entries, then reindex them cleanly.

FIELDS:
nodes_sequence: List[Dict] = dspy.InputField(
    desc="List of node dicts to be cleaned and merged"
)
node_output: List[Dict] = dspy.OutputField(
    desc="Reorganized list of nodes (deduped, merged, re-indexed A, B, C...)"
)
```

S5.2.4 Atomic decomposition

These signatures split a sentence into atomic clinical statements and check each result, re-decomposing up to two levels (`recursively_decompose_to_atomic_sentences`). We applied this recursive decomposition when generating the graphs reported in the paper; in the released code it is controlled by the `atomic_sentences` argument of the clinical-data step. The module-level `__doc__` assignments replace the class docstrings; we show the docstrings in effect.

decomposeToAtomicSentences (testing_more.py, line 102)

```
DOCSTRING:
    Given a single complex sentence from a clinical case report,
    return a list of atomic clinical sentences.

    Each atomic sentence must:
    - Contain only one clinical event, action, or state
    - Be clear, self-contained, and refer only to the patient
    - Preserve all quantitative values, agents, and time references
    - Be directly traceable to the original sentence (no hallucination)
    - Be suitable for downstream structuring into graph nodes or facts
    Do NOT summarize, group, or abstract across multiple concepts.

    If the sentence is already atomic, return it as a single-item list.

If this sentence was previously marked as non-atomic because: {context}, be especially careful to avoid that issue.

FIELDS:
sentence: str = dspy.InputField(desc="A complex clinical sentence from a case report")
context: str = dspy.InputField(optional=True, desc="Optional failure reason or context to improve decomposition")
atomic_sentences: List[str] = dspy.OutputField(desc="List of decomposed atomic sentences")
```

checkIfAtomic (testing_more.py, line 143)

```
DOCSTRING:
You are a clinical validator whose job is to strictly determine whether a sentence is atomic.

A sentence is atomic ONLY IF ALL of the following are true:
1. It describes exactly ONE clinical action, state, event, or observation.
2. It does NOT include multiple verbs or actions ("and", "then", "also", "while", "which", etc.).
3. It is NOT a compound sentence.
4. It is self-contained and clearly refers to a single patient-specific event.
5. It includes detail about the agent, value, or time if such detail is present in the source.

!If there are multiple actions, effects, causes, or interpretations -- it is NOT atomic.

Be harsh. If you're uncertain, the answer is FALSE.

Return:
- is_atomic (bool): True ONLY if the sentence is minimal and describes exactly one clinical change.
- reason (str): Explain why it is or isn't atomic.

Examples:
- "The patient underwent a PET scan." → ✓ atomic
- "The scan showed lesions and chemotherapy was started." → × not atomic (two actions)
- "He developed fever while on pembrolizumab." → × not atomic (symptom + treatment context)

FIELDS:
sentence: str = dspy.InputField()
```

```
is_atomic: bool = dspy.OutputField()
reason: str = dspy.OutputField()
```

S5.2.5 Clinical-data extraction

Applied to each node's content, with up to two retries if the output is not a dictionary. Docstring = dag_primer + node_clinical_data_instructions.

nodeClinicalDataExtract (graph_generation.py, line 282): node_clinical_data_instructions and fields

```
DOCSTRING:
Each node includes an optional structured field `clinical_data`, formatted as a dictionary with categories mapping to lists of
↳ dictionaries. Values should only be included if matched to the UMLS Metathesaurus; otherwise, omit and do not print/
↳ store it.

clinical_data = {
    "medications": [
        {
            "drug": "UMLS_CUI or string",
            "dosage": "string",
            "frequency": "string",
            "modality": "oral | IV | IM | subcutaneous | transdermal | inhaled | other",
            "start_date": "ISO8601",
            "end_date": "ISO8601 or null",
            "indication": "UMLS_CUI or string"
        }
    ],
    "vitals": [
        {
            "type": "UMLS_CUI or string",
            "value": "numeric or string",
            "unit": "string",
            "timestamp": "ISO8601"
        }
    ],
    "labs": [
        {
            "test": "UMLS_CUI or string",
            "value": "numeric or string",
            "unit": "string",
            "flag": "normal | abnormal | critical | borderline | unknown",
            "reference_range": "string",
            "timestamp": "ISO8601"
        }
    ],
    "imaging": [
        {
            "type": "UMLS_CUI or string",
            "body_part": "UMLS_CUI or string",
            "modality": "X-ray | CT | MRI | Ultrasound | PET | other",
            "finding": "string",
            "impression": "string",
            "date": "ISO8601"
        }
    ],
    "procedures": [
        {
            "name": "UMLS_CUI or string",
            "approach": "open | laparoscopic | endoscopic | percutaneous | other",
            "date": "ISO8601",
            "location": "string",
            "performed_by": "string or provider ID",
            "outcome": "string or UMLS_CUI"
        }
    ],
    "HPI": [
        {
            "summary": "string",
            "duration": "string or ISO8601",
            "onset": "string or ISO8601",
            "progression": "gradual | sudden | fluctuating | unknown",
            "associated_symptoms": ["UMLS_CUI or strings"],
            "alleviating_factors": ["UMLS_CUI or strings"],
            "exacerbating_factors": ["UMLS_CUI or strings"]
        }
    ],
}
```

```

"ROS": [
  {
    "system": "constitutional | cardiovascular | respiratory | GI | GU | neuro | psych | etc.",
    "findings": ["UMLS_CUI or strings"]
  }
],
"functional_status": [
  {
    "domain": "mobility | cognition | ADLs | IADLs | speech | hearing | vision",
    "description": "string",
    "score": "numeric (if validated scale used)",
    "scale": "Barthel Index | MMSE | MoCA | other"
  }
],
"mental_status": [
  {
    "domain": "orientation | memory | judgment | mood | speech",
    "finding": "UMLS_CUI or string",
    "timestamp": "ISO8601"
  }
],
"social_history": [
  {
    "category": "smoking | alcohol | drug use | housing | employment | caregiver | support",
    "status": "current | past | never | unknown",
    "description": "string"
  }
],
"allergies": [
  {
    "substance": "UMLS_CUI or string",
    "reaction": "UMLS_CUI or string",
    "severity": "mild | moderate | severe | anaphylaxis",
    "date_recorded": "ISO8601"
  }
],
"diagnoses": [
  {
    "code": "ICD10 or SNOMED or UMLS_CUI",
    "label": "string",
    "status": "active | resolved | historical | suspected",
    "onset_date": "ISO8601 or null"
  }
]
]

FIELDS:
content: str = dspy.InputField(desc="Narrative clinical content from a node")
atomic_sentences: list[str] = dspy.InputField(desc="List of atomic-level clinical statements derived from the node content")
clinical_data: dict = dspy.OutputField(desc="Structured clinical data dictionary with fields like medications, labs, imaging,
↳ etc.")

```

S5.2.6 Edge construction

Applied once to the ordered node list. Docstring = dag_primer + edge_instructions. The predictor is built as `dspy.Predict(edgeConstruct, examples=edge_fewshot_examples)` with the three demonstrations below.

`edgeConstruct (graph_generation.py, line 277): edge_instructions and fields`

```

DOCSTRING:
Each edge represents a change from one node to another. There should be one edge in between every pair of adjacent nodes.

Guidelines for edges:
- Create edges only when there is a clear clinical progression or change between nodes.
- Maintain narrative or logical order -- edges should flow from earlier to later events.
- Combine co-occurring findings into the same node, not across multiple edges.

Output format:
Return a list of edge dictionaries, in this order, each with:
- edge_id: Unique identifier (Use format "node_id_to_node_id", such that the first "node_id" is the upstream node and the
  ↳ second "node_id" is the downstream node bounding the edge)
- branch_flag: Boolean if this starts a side branch, default = True
- content: Exhaustive clinical content, include all relevant details for the given node

Optional structured field for edge-level transitions:
transition_event = {
  "trigger_type": "procedure | lab_change | medication_change | symptom_onset | interpretation | spontaneous",

```

```

    "trigger_entities": ["UMLS_CUI_1", "UMLS_CUI_2"], # e.g., C0025598 = Metformin, C0011581 = Chest Pain
    "change_type": "addition | discontinuation | escalation | deescalation | reinterpretation | resolution | progression |
    ↳ other", # Nature of the change
    "target_domain": "medication | symptom | diagnosis | lab | imaging | procedure | functional_status | vital_sign", # What
    ↳ category was affected
    "timestamp": "ISO 8601 datetime (e.g., "2025-03-01T10:00:00Z")", only include if explicitly given and can be converted to
    ↳ datetime
}

```

FIELDS:

```

node_input: list[dict] = dspy.InputField(desc="Ordered list of node dictionaries as generated by nodeConstruct")
edge_output: list[dict] = dspy.OutputField(desc="A list of edge dictionaries with edge_id, branch_flag, content, and optional
    ↳ transition event. There should be one edge in between every adjacent node.")

```

Few-shot edge demonstrations (graph_generation.py, lines 419–464)

```

edge_example_1 = dspy.Example(
    node_input=[{"node_id": "A"}, {"node_id": "B"}],
    edge_output=[{
        "edge_id": "A_to_B",
        "branch_flag": True,
        "content": "Severe headache led to CT finding of subdural hematoma.",
        "transition_event": {
            "trigger_type": "symptom_onset",
            "trigger_entities": ["C0018681", "C0038454"], # e.g., UMLS CUIs
            "change_type": "progression",
            "target_domain": "imaging"
        }
    }]
).with_inputs("node_input")

edge_example_2 = dspy.Example(
    node_input=[{"node_id": "B"}, {"node_id": "C"}],
    edge_output=[{
        "edge_id": "B_to_C",
        "branch_flag": True,
        "content": "Rash developed after antibiotics.",
        "transition_event": {
            "trigger_type": "medication_change",
            "trigger_entities": ["C0003232", "C0037274"], # Antibiotics, Rash
            "change_type": "adverse_effect",
            "target_domain": "symptom"
        }
    }]
).with_inputs("node_input")

edge_example_3 = dspy.Example(
    node_input=[{"node_id": "C"}, {"node_id": "D"}],
    edge_output=[{
        "edge_id": "C_to_D",
        "branch_flag": False,
        "content": "PET-CT showed regression of disease.",
        "transition_event": {
            "trigger_type": "interpretation",
            "trigger_entities": ["C0151744"], # Regression
            "change_type": "resolution",
            "target_domain": "diagnosis"
        }
    }]
).with_inputs("node_input")

edge_fewshot_examples = [edge_example_1, edge_example_2, edge_example_3]

```

S6 Additional Related Work

S6.1 Expansion beyond existing extraction methods

We situate Monopath DAGs within prior efforts on case report extraction and patient-journey modeling. The below table summarizes representative approaches, contrasting pipelines, outputs, and downstream utility. Whereas earlier resources focus on entity spans, citation heuristics, or free-text knowledge graphs, our method produces temporally ordered DAGs whose nodes and edges carry optional UMLS-coded structured fields.

Work	Extraction Pipeline	Output	Utility
Monopath DAGs	LLM + DSPy pipeline	DAG per patient (ordered state nodes and transition edges; optional UMLS-coded fields)	Evaluated here for reconstruction fidelity (BERTScore), annotator graph review, clustering, metastasis prediction, and physician ratings of graph-conditioned synthetic narratives
PMC-Patients (Zhao, 2023)	Regex & heuristics; citation links	Paragraph + triples	Retrieval benchmark only; no temporal or causal
CaseReportBench (Zhang, 2025)	Human BIO annotation	Entity spans	High-quality labels; no temporal or relation structure
Patient-Journey KGs (Al Khatib, 2025)	Prompt-based LLM; node merging	Multi-relational KG per patient	Structural compliance evaluation; no clinical validation

Comparison with PMC-Patients. While both our work and Zhao et al. (PMC-Patients) leverage case reports, the approaches differ fundamentally.

Scope: PMC-Patients is designed as a retrieval benchmark using free-text patient summaries and citation links, whereas our work extracts an ordered sequence of patient states and the transitions between them.

Granularity: PMC-Patients preserves paragraphs, but Monopath DAGs break narratives into machine-readable node-edge pairs, with optional timestamps and UMLS-coded fields; we use them for clustering and for conditioning synthetic narratives.

Evaluation: Zhao et al. report retrieval performance; we instead measure reconstruction fidelity (BERTScore), compare clustering of graph and text encodings (silhouette and Calinski-Harabasz), and collect blinded physician ratings of graph-conditioned and control synthetic narratives.

Use-case: In this paper we use the graphs to condition synthetic case narratives and to group cases by trajectory. Other uses, such as trajectory-aware search, have not been evaluated.

S7 Computational Resources

The pipeline is implemented in Python and runs on standard research hardware. Experiments used a single NVIDIA A100 GPU (40GB) with 64GB RAM, though CPU-only execution is possible with longer runtimes. End-to-end processing takes about 2–3 minutes per case report, making the approach accessible to academic and clinical groups.

S8 Physician evaluation: analyses

Three physicians rated both versions of the cases they saw (227 ratings across 76 cases; 97 pairs across 63 cases); a fourth rated a single narrative and is excluded. The primary analysis is a linear mixed-effects model for each question, with a fixed effect for condition (graph-conditioned vs. control) and crossed random intercepts for physician and case, fit by REML; p values are Holm-adjusted across the five questions. Sensitivity analyses: (i) per-physician mean paired differences with bootstrap 95% CIs over cases (5,000 resamples); (ii) the mixed model refit with each physician excluded; (iii) paired differences averaged within case and tested with the Wilcoxon signed-rank test (Holm-adjusted), with bootstrap CIs; (iv) Spearman correlation between physicians on shared ratings. Physicians are anonymized as A–C.

Table S8.1: Primary analysis: mixed-model estimate of graph-conditioned minus control rating.

Question	Estimate	95% CI	Holm p
Clinical validity	0.21	[−0.05, 0.48]	0.117
Timeline validity	0.46	[0.18, 0.73]	0.005
Clinical actionability	0.45	[0.17, 0.72]	0.005
Clinically safe action	0.45	[0.21, 0.68]	0.001
Language appropriateness	0.28	[0.03, 0.53]	0.058

Table S8.2: Per-physician mean paired difference (graph-conditioned – control) with bootstrap 95% CI. Pairs: A = 42, B = 28, C = 27.

Question	Physician A	Physician B	Physician C
Clinical validity	0.21 [−0.17, 0.57]	0.57 [0.11, 1.07]	−0.22 [−0.85, 0.44]
Timeline validity	0.33 [−0.05, 0.71]	0.75 [0.18, 1.32]	0.33 [−0.30, 0.96]
Clinical actionability	0.50 [0.17, 0.86]	0.64 [0.07, 1.21]	0.22 [−0.52, 0.96]
Clinically safe action	0.48 [0.17, 0.76]	0.57 [0.14, 1.04]	0.37 [−0.26, 1.00]
Language appropriateness	0.32 [−0.02, 0.68]	0.46 [−0.04, 0.96]	0.04 [−0.59, 0.70]

Table S8.3: Leave-one-physician-out mixed models: estimate [95% CI]; Holm p , with the named physician excluded.

Question	Without A	Without B	Without C
Clinical validity	0.19 [−0.19, 0.57]; 0.346	0.08 [−0.25, 0.42]; 0.650	0.36 [0.09, 0.63]; 0.008
Timeline validity	0.47 [0.07, 0.86]; 0.084	0.35 [0.02, 0.68]; 0.159	0.53 [0.24, 0.83]; 0.001
Clinical actionability	0.40 [−0.02, 0.81]; 0.183	0.36 [0.01, 0.70]; 0.159	0.55 [0.27, 0.83]; <0.001
Clinically safe action	0.43 [0.08, 0.79]; 0.084	0.40 [0.10, 0.71]; 0.051	0.48 [0.24, 0.71]; <0.001
Language appropriateness	0.26 [−0.11, 0.62]; 0.346	0.16 [−0.16, 0.47]; 0.650	0.39 [0.13, 0.65]; 0.008

Table S8.4: Case-level analysis: paired differences averaged within case, Wilcoxon signed-rank test.

Question	Cases	Mean difference [95% CI]	Holm p
Clinical validity	63	0.25 [−0.05, 0.56]	0.055
Timeline validity	63	0.48 [0.18, 0.79]	0.019
Clinical actionability	63	0.50 [0.21, 0.79]	0.006
Clinically safe action	63	0.46 [0.20, 0.70]	0.002
Language appropriateness	62	0.33 [0.08, 0.58]	0.040

Table S8.5: Agreement between physicians on shared ratings (Spearman ρ). Shared ratings: A–B = 28; A–C = 31; B–C = 44.

Question	A–B	A–C	B–C
Clinical validity	0.03	0.14	0.18
Timeline validity	0.34	−0.01	0.37
Clinical actionability	−0.10	−0.07	0.32
Clinically safe action	0.21	0.09	0.22
Language appropriateness	0.12	−0.02	0.22

S9 Clustering and metastasis analyses

Settings. Embeddings: emilyalsentzer/Bio_ClinicalBERT, final-layer [CLS] vector. Graph: mean of node-text embeddings. Case text: the case-presentation section of each PMC article, tokenized and split into consecutive non-overlapping 510-token windows, each wrapped in [CLS]/[SEP]; the report embedding is the mean of window embeddings (mean length 1,137 tokens, median 967; 73% exceed 512). Reduction: PCA to 10 components (no whitening); UMAP to 10 components (15 neighbors, minimum distance 0.1, random state 42). Clustering: K-means (scikit-learn, random state 4), $k \in [2, 10]$ selected by silhouette, patients in sorted order. Metastasis analyses use the 484 patients with recorded status (73% metastatic); metastasis status was annotated manually for each report: Calinski–Harabasz (CH) index with patients partitioned by status, and L2-regularized logistic regression ($C = 0.1$, standardized inputs), 5-fold stratified cross-validation repeated 10 times, AUC from pooled out-of-fold probabilities. 95% CIs: patient-level bootstrap (1,000 resamples for CH, with K-means re-fit on each resample; 2,000 for AUC). **Edge-aware variants:** nodes + edge text, [mean node embedding; mean edge-description embedding]; path as text, the path written out in order (“State 1: ... Transition: ... (trigger; change; domain). State 2: ...”) and embedded like the case text; trajectory shape, [first; last; mean node; mean consecutive change].

Table S9.1: Cluster CH at fixed k , graph (nodes) vs. original text.

Space	k	Graph	Original text
Original	2	136.5	77.6
Original	3	96.3	55.0
Original	4	71.7	44.8
Original	5	59.7	41.2
Original	6	50.6	37.1
PCA	2	224.2	147.2
PCA	3	167.3	109.8
PCA	4	127.2	92.9
PCA	5	108.2	89.8
PCA	6	95.2	79.4
UMAP	2	738.4	357.9
UMAP	3	697.4	309.4
UMAP	4	629.9	306.5
UMAP	5	576.8	296.6
UMAP	6	539.6	296.6

Table S9.2: Stability across six patient orderings (sorted and five random): selected k and CH mean (range).

Space	Representation	Selected k	CH
Original	Graph	[2]	136.4 (136.4–136.5)
Original	Original text	[2]	77.6 (77.5–77.6)
PCA	Graph	[2]	224.1 (223.9–224.2)
PCA	Original text	[2]	147.3 (147.2–147.4)
UMAP	Graph	[2]	748.5 (726.8–778.6)
UMAP	Original text	[2]	356.3 (350.4–366.8)

Adding edge embeddings to nodes. To test whether edges add information, we also embedded each edge description with the same model and represented each graph by the mean over all node and edge embeddings, either weighting every element equally (element-weighted) or giving nodes and edges equal total weight (50:50). We repeated the metastasis classifier over 10 cross-validation splits and three regularization strengths ($C = 0.01, 0.1, 1.0$). Adding edges to nodes raised AUC in all 60 runs (both weightings), by 0.01–0.03; at $C = 0.1$ the patient-level bootstrap difference (nodes + edges minus nodes) was +0.024 [−0.006, +0.055]. With edges, the graph reached the AUC of the full report text, whereas edges alone were slightly weaker than nodes. Adding edges lowered cluster compactness (CH at $k = 2$ in the original space: nodes

Table S9.3: Cluster CH at $k = 2$ with bootstrap 95% CI, and difference from original text. The trajectory-shape UMAP value reflects UMAP on a 3,072-dimensional concatenation and should not be interpreted.

Space	Representation	CH [95% CI]	Difference vs. text
Original	Nodes (primary)	136.5 [118.7, 155.8]	58.9 [33.6, 78.9]
Original	Nodes + edge text	65.2 [40.0, 76.4]	-12.4 [-42.1, 2.3]
Original	Path as text	45.1 [23.6, 52.2]	-32.4 [-55.5, -22.1]
Original	Trajectory shape	81.6 [50.1, 88.9]	4.0 [-30.7, 15.9]
Original	Original text	77.6 [68.8, 90.1]	
PCA	Nodes (primary)	224.2 [197.9, 257.7]	77.0 [41.2, 114.0]
PCA	Nodes + edge text	114.1 [74.4, 136.5]	-33.1 [-75.0, -1.8]
PCA	Path as text	90.2 [69.1, 104.1]	-57.0 [-86.2, -32.9]
PCA	Trajectory shape	189.0 [108.0, 206.5]	41.8 [-43.1, 68.8]
PCA	Original text	147.2 [127.2, 170.6]	
UMAP	Nodes (primary)	738.4 [649.5, 841.1]	380.5 [284.5, 493.1]
UMAP	Nodes + edge text	374.4 [262.3, 438.8]	16.5 [-109.8, 92.7]
UMAP	Path as text	221.8 [191.8, 257.5]	-136.1 [-194.5, -84.3]
UMAP	Trajectory shape	5633.6 [5329.3, 5951.9]	5275.7 [4967.9, 5596.4]
UMAP	Original text	357.9 [318.2, 407.9]	

Table S9.4: Metastasis status: cross-validated AUC and CH with patients partitioned by status (original space), with differences from original text.

Representation	AUC [95% CI]	Δ AUC	CH by status	Δ CH
Nodes (primary)	0.732 [0.677, 0.782]	-0.038 [-0.091, 0.014]	10.6 [6.1, 19.3]	3.6 [-1.6, 10.7]
Nodes + edge text	0.755 [0.704, 0.802]	-0.014 [-0.067, 0.038]	7.8 [5.0, 14.1]	0.8 [-3.2, 5.5]
Path as text	0.687 [0.632, 0.741]	-0.082 [-0.135, -0.027]	4.7 [3.8, 8.4]	-2.3 [-5.2, 0.2]
Trajectory shape	0.706 [0.652, 0.757]	-0.064 [-0.119, -0.007]	4.2 [3.1, 8.2]	-2.9 [-6.3, 0.4]
Original text	0.769 [0.722, 0.815]		7.0 [5.3, 11.5]	

136.5, nodes + edges 100.9; difference -35.6 [-49.3, -21.2]).

Table S9.5: Metastasis status: cross-validated AUC (mean over 10 cross-validation splits) by regularization strength C . Nodes + edges exceeded nodes in 60/60 runs.

Representation	$C = 0.01$	$C = 0.1$	$C = 1.0$
Nodes	0.756	0.718	0.697
Edges	0.729	0.695	0.673
Nodes + edges (element-weighted)	0.766	0.738	0.723
Nodes + edges (50:50)	0.769	0.742	0.727
Original text	0.766	0.745	0.728